Future Directions: Integrating Generative AI with Traditional Big Data Pipelines

CONVERGE—Validation-Guided Language-Model Recommendations for Spark Data Cleaning

Divyanshi Vashistha
[Department / Institution]
[City, Country]
divyanshivashistha@gmail.com

***Abstract*—Generative language models can supply the semantic judgement that rule-based data cleaning lacks, but wiring their output directly into a big data engine turns every hallucination into a silent, scaled corruption of data. We present CONVERGE, a validation-guided architecture for integrating language models into Apache Spark pipelines in which *AI recommends, a deterministic validator decides, and Spark executes*. The model sees only a bounded, Spark-computed evidence bundle and may only emit typed recommendations from a closed operation catalog; an independent validator cross-checks every recommendation against the same evidence before an immutable, fail-fast plan is executed. We evaluate CONVERGE against a deterministic Spark-only baseline and an unvalidated AI-only pipeline on six controlled-corruption scenarios with per-cell ground truth, using a locally deployed 3B-parameter model (Qwen2.5-Coder) on commodity hardware (54 end-to-end runs), a twelve-class hallucination-injection study, and a scalability study from 10^2 to 10^6 rows. AI is necessary for semantic repairs (date-format recall $0 \rightarrow 1$), but unvalidated AI also fabricated issues and modified up to 45 clean cells in a run—damage that conventional before/after quality checks reported as complete success. The validation boundary reduced collateral damage by 71–88%, rejected nine of eleven checkable hallucination classes (preventing deleted records, smuggled parameters and run aborts), and cost under one millisecond per plan with a prompt size independent of data volume. A recommendation-level audit also exposes the limits of validation: evidentially consistent but semantically wrong repairs, catalog-level evidence conflation, and an evidence-ambiguity defect, each of which we characterise and for which we propose concrete remedies and future research directions.**

***Index Terms*—big data pipelines, Apache Spark, generative AI, large language models, data cleaning, hallucination, validation, data quality, AI safety**

I. Introduction

Traditional big data pipelines built on MapReduce [1] and Apache Spark [2], [3] are engineered around a simple contract: transformations are *deterministic*, *declarative* and *auditable*. A Spark job that trims whitespace or removes duplicate rows will do exactly the same thing on every run, and its behaviour can be read off its logical plan. Data cleaning, however, has never fit this contract comfortably. Choosing the canonical spelling of a category, the target format for a date column, or an imputation strategy for missing salaries requires *semantic judgement* that rule-based systems either hard-code or delegate to humans [4]–[6].

Large language models (LLMs) appear to close this gap. They can read a column profile and propose that `ENGINEERING`, `engineering` and `Engineering` are the same department, or that `15/03/2024` should be rewritten as `2024-03-15` [7], [8]. Yet integrating a generative model into a production data pipeline imports a failure mode that pipelines were never designed for: *hallucination* [9], [10]. A model may invent an operation, reference a column that does not exist, fabricate a category value, misstate a statistic, or confidently recommend a destructive transformation. When the model's output is wired directly to an execution engine—as in text-to-SQL and tool-using agents [11]–[13]—each such error becomes a silent, irreversible corruption of data at scale.

This paper studies a deliberately conservative answer to the question *how should generative AI be integrated into traditional big data pipelines?* We present **CONVERGE**, a validation-guided architecture in which *AI recommends, a deterministic validator decides, and Spark executes*. The LLM never sees raw rows, never emits code, and never touches the execution engine. Instead, it receives a bounded, Spark-computed evidence bundle and returns typed, data-only *recommendations* drawn from a closed operation catalog. An independent validator cross-checks every recommendation against the same evidence before anything executes, and a frozen execution plan is applied by Spark under fail-fast semantics.

The central empirical question is whether this boundary matters. We therefore compare three arms under controlled, ground-truth conditions: a deterministic SPARK_ONLY baseline, an AI_ONLY arm in which the same model's recommendations are executed after only a structural filter, and the full CONVERGE pipeline. All arms share the same profiler, executor, prompt, and evaluator, so the *only* variable is the decision architecture.

Contributions. This paper makes the following contributions:

1) **Architecture.** We formalise and implement a validation-guided integration pattern for LLMs in Spark pipelines, with five explicit safety properties (bounded evidence, closed action space, independent validation, human/policy approval boundary, immutable fail-fast execution) and a ten-state auditable pipeline state machine (§IV).
2) **Evaluation methodology.** We contribute a controlled-corruption benchmark with per-cell ground-truth manifests and strategy-agnostic metrics (repair recall, false-modification rate, row loss) that separate *observed* quality

signals from *correctness* (§V).

3) **Hallucination taxonomy and causal study.** We define a twelve-class taxonomy of recommendation-level hallucinations and inject each into otherwise-correct plans, isolating the causal effect of the validator from model stochasticity (§VI-D).
4) **Empirical findings.** Using a locally deployed small language model (Qwen2.5-Coder 3B [14]), we report end-to-end repair quality, reliability, and latency across six scenarios, and show that the AI/validation control plane is independent of data volume while Spark carries the row-proportional work (§VI).
5) **Limits and future directions.** We characterise precisely what validation *cannot* catch—semantically wrong but evidentially consistent recommendations and information-theoretically unrecoverable repairs—and derive a research agenda for AI-augmented big data systems (§VII–IX).

II. Background and Related Work

A. Data Cleaning in Big Data Systems

Data cleaning is a long-standing database problem spanning error detection and repair [4]–[6], [15]. Constraint-based methods such as conditional functional dependencies [16] provide formal repair semantics but require curated rules. Probabilistic systems such as HoloClean [17] unify signals into a factor graph, while ActiveClean [18] and Raha [19] reduce human effort through learning. Abedjan et al. [20] showed that no single detector dominates across real datasets. At the pipeline level, Deequ [21] and TFX data validation [22] introduced declarative "unit tests for data" over Spark. CONVERGE inherits this philosophy—quality checks are deterministic Spark computations—but applies validation to *model-proposed actions* rather than to the data alone.

B. Language Models for Data Wrangling

Foundation models can perform entity matching, imputation, and error detection with few or no labelled examples [7], [8], [23]. Interactive tools such as Wrangler [24] earlier showed the value of *suggesting* transformations rather than applying them. LLM-based text-to-SQL [11], [25] and tool-using agents [12], [13], [26] go further, generating executable programs. These lines of work typically measure task accuracy; few ask what happens when an incorrect suggestion is executed against a production dataset. Our AI_ONLY arm models precisely that integration style.

C. Hallucination and LLM-Integrated System Safety

Hallucination—fluent output unsupported by the input or by reality—is a pervasive property of generative models [9], [10]. Retrieval augmentation [27] and constrained decoding [28] reduce but do not eliminate it. Indirect prompt injection [29] further shows that LLM-integrated applications inherit adversarial risks from any text the model reads. From a systems-security perspective, Saltzer and Schroeder's principles of *least privilege* and *complete mediation* [30] suggest that an untrusted component should never hold execution authority. CONVERGE operationalises these principles: the model holds *proposal* authority only, and every proposal is mediated by a validator with no AI dependency. This also mitigates the "hidden technical debt" of entangled ML components [31] and addresses concrete safety concerns such as negative side effects [32].

D. Positioning

Prior work either (i) cleans data deterministically without semantic judgement, or (ii) uses LLMs for semantic judgement without an execution-safety boundary. To our knowledge, CONVERGE is the first to evaluate, against per-cell ground truth and a controlled hallucination taxonomy, an architecture that *decouples* generative recommendation from big data execution through evidence-based validation.

III. Problem Formulation and Threat Model

Let D be a tabular dataset with schema $C = \{c_1, \ldots, c_m\}$ and n rows. A deterministic Spark profiler computes $\pi(D) = (P, E)$, where P is an aggregate profile (per-column dtype, semantic type, null counts, distinct counts, numeric quantiles, date-pattern histograms) and $E = \{e_1, \ldots, e_k\}$ is a set of *evidence items*. Each $e = (\tau, C_e, \sigma)$ has an issue type $\tau \in \mathcal{T}$ (missing values, duplicates, categorical inconsistency, whitespace, date-format inconsistency, outlier candidate), a column set $C_e \subseteq C$, and bounded supporting data σ (e.g., a variant histogram). Crucially, $|E|$ and $|\sigma|$ are bounded by configuration constants, not by n.

An operation catalog $\mathcal{O}$ defines, for every permitted operation o, a safety tier $t(o) \in \{\text{SAFE}, \text{REVIEW}, \text{FINDING}\}$, compatible semantic types $S(o)$, a parameter schema $\Theta(o)$, and required evidence types $R(o) \subseteq \mathcal{T}$. A separate forbidden set $\mathcal{F}$ (e.g., `DELETE_COLUMN`, `ARBITRARY_CODE`, `JOIN`) is recognised and always refused.

A language model M maps $(P, E, \mathcal{O})$ to a list of recommendations $r = (o, C_r, \theta, \rho, \text{refs})$: an operation name, target columns, parameters, a free-text reason, and evidence references carrying *claimed values*. We treat M as *untrusted*: any field of r may be wrong.

Hallucination classes. We say r is *hallucinated* if it violates any of: (H-op) $o \in \mathcal{O} \setminus \mathcal{F}$; (H-col) $C_r \subseteq C$; (H-type) $\forall c \in C_r : \text{sem}(c) \in S(o)$; (H-param) θ conforms to $\Theta(o)$ with no extra keys; (H-evid) $\exists e \in E$ with $\tau_e \in R(o)$ and $C_r \subseteq C_e$ (exact equality for row-level operations); (H-claim) every claimed value equals the corresponding σ entry; (H-value) every raw value named in θ occurs in σ. These are *checkable* hallucinations. A recommendation may also be *semantically wrong*—e.g., mapping a real variant of "Marketing" to "Sales"—while satisfying all checkable predicates; we show in §VI-D that no evidence-based validator can detect this class.

Safety properties. CONVERGE is designed to guarantee:

P1 *Bounded exposure*: M observes only $(P, E, \mathcal{O})$, never rows of D, paths, or engine handles.
P2 *Closed action space*: outputs are typed data; no field is ever evaluated as code.

P3 *Complete mediation*: no r reaches execution unless the validator $V(r; P, E)$ returns an executable status.

P4 *Monotone approval*: approval policies may only narrow V's verdict, never widen it.

P5 *Immutable, fail-fast execution*: the executor receives a frozen plan, applies it in order, re-validates parameters, and writes no output if any operation fails.

IV. The CONVERGE Architecture

Fig. 1 shows the end-to-end architecture. Its organising principle is a strict separation of *authority*: Spark is authoritative for facts, the LLM is authoritative for nothing, the validator is authoritative for admissibility, and the approval boundary is authoritative for consent.

A. Spark Profiler and Bounded Evidence

The profiler (`services/spark/profiler.py`) is the only component that reads D. For each column it classifies a *semantic type* (integer, float, string, date-like string, boolean) separately from the physical Spark dtype, computes null statistics, distinct counts, IQR outlier bounds, case/whitespace-normalised *variant groups* for categorical columns, and a histogram of detected date patterns. Identifier-like columns (an `id` name pattern and $\geq 90\%$ distinct values) are excluded from duplicate criteria. From the profile, it emits one evidence item per detected issue; nothing is fabricated for absent issues. Evidence carries only issue-scoped raw values—e.g., the variant histogram $\{$`Engineering`:5, `engineering`:1, `ENGINEERING`:1$\}$—capped by `MAX_EVIDENCE_DISTINCT_VALUES`$= 30$ and `MAX_EVIDENCE_SAMPLE_ROWS`$= 10$. Before prompting, the prompt builder additionally *redacts* unscoped raw fields (per-column samples and top-value lists), so that, for example, personal names in an unflagged column are never sent to the model. This realises P1 and has a second, decisive consequence: the prompt size is $O(m + |E|)$ and independent of n (verified in §VI-E).

B. Operation Catalog and Safety Tiers

Table I lists the eight P0 operations. Operations are declarative, parameterised, and implemented exclusively with the Spark DataFrame API—no `spark.sql()`, `eval`, or dynamic dispatch on parameter content. The SAFE tier covers value-preserving normalisations; REVIEW covers operations that synthesise values or change row counts and are capped at NEEDS_REVIEW regardless of evidence; FINDING operations can never execute. The same registry generates the prompt's operation list, the validator's rules, and the traceability map, so the three cannot drift apart (ADR-002).

TABLE I
Operation catalog (P0). Tier caps the best achievable verdict.

Operation	Tier	Parameters	Required evidence
`TRIM_WHITESPACE`	safe	–	whitespace
`NORMALIZE_CASE`	safe	case∈{lower,upper,title}	categorical ∨ ws
`STANDARDIZE_CATEGORY`	safe	mapping: raw→canonical	categorical
`STANDARDIZE_DATE_FORMAT`	safe	fmt∈{ISO, D/M/Y, M/D/Y}	date-format
`FILL_MISSING`	review	mean|median|mode|const	missing
`CONVERT_DTYPE`	review	int|float|str|bool	–
`REMOVE_DUPLICATES`	review	criteria = target cols	duplicates (exact)
`FLAG_OUTLIER`	finding	–	outlier

Forbidden: `DELETE_COLUMN`, `DELETE_DATASET`, `JOIN`, `PIVOT`, `MERGE`, `ARBITRARY_ROW_FILTER`, `ARBITRARY_CODE`, `SCHEMA_RESTRUCTURING`

C. AI Provider and Prompt Contract

Providers implement a four-member interface: `provider_name`, `model_name`, `prompt_version`, and `generate_recommendations(ProfileResult)` → `list[Recommendation]`. The signature itself enforces P1 and P2: there is no parameter through which raw data could enter and no return channel for code. Two implementations exist: a remote Gemini provider [33] using native response-schema enforcement, and a local Ollama provider using JSON-schema-constrained decoding [28] at temperature 0. Both share one versioned system prompt (`recommendation-prompt-v1`) stating eight rules: Spark evidence is authoritative; do not invent facts; do not calculate; only use catalog operations; never produce code; never propose destructive operations; cite evidence with checkable claimed values; and return *fewer* recommendations when evidence is insufficient. Output is parsed strictly with Pydantic; any malformed item fails the stage with a typed `AIProviderResponseError` rather than being silently repaired. Note that `operation` is deliberately typed as a free string rather than an enum, so that an invented operation *constructs* successfully and is rejected by the validator with an auditable explanation instead of an opaque parse error.

D. Independent Validator

The validator (Algorithm 1) is constructed once per profile and has no dependency on the AI provider, the executor, or the orchestrator (ADR-003); it can be tested in isolation against hand-built recommendations. Rules are evaluated in order and short-circuit on the first failure, and every check is recorded with a human-readable message, giving a complete audit trail. The evidence rules are the core novelty: they do not trust the model's `evidence_refs`, but independently search E for an item of a required type covering the targets. For row-level operations the criteria set must equal the profiler's criteria *exactly*, preventing the model from asserting duplicate support for a column set Spark never verified. Claimed numeric values are compared with tolerance 10^{-6}, and every raw key of a category mapping must occur in the evidence's variant histogram.

E. Approval Boundary and Pipeline State Machine

The orchestrator drives an explicit state machine: INGESTED → PROFILED → AI_ANALYZED → VALIDATED → AWAITING_APPROVAL → EXECUTING → CLEANED → STATS_COMPUTED → INSIGHTS_GENERATED → COMPLETED, with FAILED reachable from any stage. Every transition is timestamped and persisted. The pipeline *hard-pauses* at AWAITING_APPROVAL; nothing executes without an explicit decision per recommendation. In interactive use a human

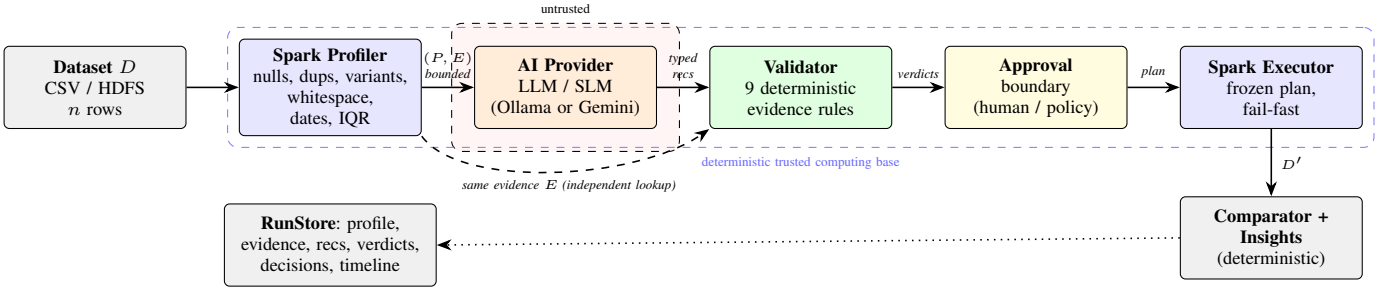


Fig. 1. CONVERGE architecture. The language model (red, untrusted) sees only the bounded profile/evidence (P, E) and emits typed recommendations. The validator independently re-reads the *same* Spark evidence; only admissible, approved recommendations become an immutable execution plan. Measurement is deterministic and external to the AI.

Algorithm 1 Evidence-based validation $V(r; P, E)$

Input: recommendation $r = (o, C_r, \theta, \cdot, \text{refs})$; profile P; evidence E

if $o \in \mathcal{F}$ **or** $o \notin \mathcal{O}$ **then return** REJECTED ▷ R1, R8
end if
if $C_r = \emptyset$ **or** $C_r \not\subseteq C$ **then return** REJECTED ▷ R2
end if
if $\neg\text{rowlevel}(o) \wedge \exists c \in C_r : \text{sem}(c) \notin S(o)$ **then return** REJECTED
end if
if θ has unknown keys, missing/ill-typed required keys, or values $\notin$ allowed **then return** REJECTED
end if
if $R(o) \neq \emptyset$ **then**
  $M \leftarrow \{e \in E : \tau_e \in R(o) \wedge (\text{rowlevel}(o)\,?\, C_e{=}C_r : C_r{\subseteq}C_e)\}$
  if $M = \emptyset$ **then return** REJECTED ▷ R6: no Spark evidence
  end if
  for all $(k, v) \in$ refs.claimed with $k \in \sigma_e$, $e \in M$ **do**
    if $|v - \sigma_e[k]| > \epsilon$ **then return** REJECTED ▷ R7
    end if
  end for
  if $o =$ STANDARDIZE_CATEGORY $\wedge$ keys(θ.map) $\not\subseteq \bigcup_{e \in M}$ variants(e) **then return** REJECTED
  end if
end if
if $t(o) =$ FINDING **then return** NOT_EXECUTABLE
end if
return $t(o) =$ SAFE ? APPROVED_AUTO : NEEDS_REVIEW

approves NEEDS_REVIEW items; in experiments a documented policy approves exactly what the validator deemed executable. P4 is enforced twice: `submit_decision` raises `RecommendationNotEligibleError` for an ineligible approval, and the experiment adapter ANDs any policy with the validator's verdict so a misconfigured policy fails safe.

F. Spark Executor

Approved recommendations are compiled into a frozen `ExecutionPlan`, ordered by a fixed technical precedence (dtype conversion → imputation → text/category normalisation → date formatting → deduplication) so that, e.g., trimming precedes duplicate detection. The executor applies operations exactly in plan order, re-checks every parameter (defence in depth), counts affected rows, and on the first failure marks remaining operations SKIPPED and writes no output (P5). Partial cleaning is never published.

G. Deterministic Comparison and Insights

After execution the cleaned dataset is re-profiled with the *same* profiler and compared per issue as *resolved*, *improved*, *unchanged*, *worsened*, or *introduced*. A traceability map links each executed recommendation to the issues it targeted, and a deterministic insight generator summarises outcomes in templated natural language. No model is consulted after execution: the system never asks the AI to grade its own work (ADR-007, ADR-009).

H. Design Rationale

Three design decisions merit justification. *(i) Recommendation, not generation of code.* Generating Spark or SQL code would give the model an unbounded action space; restricting it to a catalog makes every possible action enumerable, testable, and reviewable. *(ii) Validation against evidence, not against the model's claims.* Self-consistency checks on model output are vulnerable to correlated errors; checking against an independent deterministic artefact breaks that correlation. *(iii) A narrow deterministic baseline.* The SPARK_ONLY arm applies only the two remediations whose correct action is fully determined by evidence (trim whitespace; drop profiler-verified duplicates), and *refuses* to invent heuristics (e.g., "most frequent variant is canonical") for the rest. Determinism is not correctness; the baseline therefore marks exactly the boundary where semantic judgement—and hence AI—becomes necessary.

V. Experimental Methodology

A. Research Questions

RQ1 Does AI-assisted cleaning improve repair quality over a deterministic Spark-only baseline?
RQ2 Does the validation boundary reduce hallucination-driven errors relative to unvalidated AI, and at what cost?
RQ3 Which classes of hallucination can and cannot be contained by evidence-based validation?
RQ4 How do latency, reliability, and the control-plane footprint behave, and how do they scale with data volume?

B. Experimental Arms

All three arms share the profiler, the operation catalog, the executor, the prompt, the provider, and the evaluator

TABLE II
CONTROLLED CORRUPTION SCENARIOS (SEED 42).

ID	Corruption	#	Target / example
S1	Whitespace injection	3	`name`: "␣␣Alice␣Martin␣␣"
S2	Exact duplicate rows	2	whole row, new `_row_id`
S3	Categorical case variants	3	`department`: "mARKETING"
S4	Missing values	2	`salary`: empty
S5	Mixed date formats	3	`hire_date`: "15/01/2022"
S6	All five combined	13	mixed (3+2+3+2+3)

(ADR-007). They differ only in the decision architecture. **(A) SPARK_ONLY**: deterministic policy; `TRIM_WHITESPACE` for whitespace evidence and `REMOVE_DUPLICATES` on profiler-established criteria, all other issues explicitly skipped with a recorded reason. It never imports the AI package (enforced by an import-scanning test). **(B) AI_ONLY**: the LLM's recommendations are executed in the model's own order after only a *structural* filter (operation registered, not forbidden, not finding-only, has a target). The validator module is never imported (also enforced by test). The executor's own parameter re-checks still apply, so this arm is a realistic "LLM drives a safe engine" integration rather than a strawman with arbitrary code execution. **(C) CONVERGE**: the full orchestrator with validator and the monotone approval policy. To avoid self-grading, the adapter stops at CLEANED; all measurement is external.

C. Controlled Scenarios and Ground Truth

A 15-row, 7-column clean reference (employee records: `_row_id`, `name`, `department`, `city`, `hire_date`, `salary`, `rating`) is corrupted by a seeded deterministic generator (seed 42) into six scenarios (Table II). Each corruption is recorded in a manifest with row id, column, clean value and corrupted value, and the scenario's SHA-256 checksum is verified before every run. A deliberately small reference makes every cell auditable; scale is studied separately (E3).

D. Metrics

Let $\mathcal{K}$ be the set of injected cell corruptions, $\mathrm{out}(i,c)$ the output value of row i, column c (joined on `_row_id`), and $\mathrm{clean}(i,c)$ the reference value.

$$\text{Repair recall} = \frac{|\{(i,c) \in \mathcal{K} : \mathrm{out}(i,c) = \mathrm{clean}(i,c)\}|}{|\mathcal{K}|}, \quad (1)$$

$$\text{FMR} = \frac{|\{(i,c) \notin \mathcal{K} : \mathrm{out}(i,c) \neq \mathrm{clean}(i,c)\}|}{|\mathrm{rows}_{\mathrm{out}}| \cdot m' - |\mathcal{K}|}, \quad (2)$$

where comparison is exact (case and whitespace significant) and m' excludes `_row_id`. A corrupted cell is *incorrectly repaired* if its output differs from both the clean and the corrupted value. For duplicates we count injected ids removed and *original rows lost*, i.e., clean-reference ids absent from the output—the most severe collateral damage, since FMR covers surviving rows only. Repair *precision* is not reported because intended modifications are not observable from outputs. We additionally report the comparator's before/after issue counts (an *observed* signal, never merged with correctness, ADR-009), run success rate, recommendation consistency (mean pairwise Jaccard over (o, C_r) sets across repetitions), and wall-clock times measured externally with `perf_counter` at three non-overlapping boundaries (strategy execution, evaluation, total).

E. Model, Platform, and Protocol

The language model is **Qwen2.5-Coder 3B** [14] (4-bit quantised, 1.9 GB) served locally by Ollama 0.34 with JSON-schema-constrained decoding and temperature 0; it is classified SLM/LOCAL. A local small model reflects a realistic privacy-preserving deployment in which no data or metadata leaves the cluster, and is the harder setting for structured-output reliability. All experiments run on a single commodity laptop (AMD Ryzen 3 5300U, 4 cores/8 threads, 7.5 GB RAM, no GPU). Spark 3.5.1 runs in `local[*]` mode on OpenJDK 17 and Python 3.12 inside Ubuntu 24.04 (WSL2, 3.5 GB memory limit); the model is served by Ollama on the Windows host, so model inference and Spark share the same four physical cores. Experiments E1–E3 were executed sequentially to avoid cross-experiment interference. The default provider timeout is 600 s. Each (scenario, configuration) cell is repeated $R = 3$ times, giving 6 scenarios $\times$ 3 arms $\times$ 3 repetitions $= 54$ attempted runs in E1. Every run is preserved, including failures, with checksums, provider, model, prompt version and package version.

F. Experiments

E1 (end-to-end matrix) executes all arms on S1–S6 with the live model. **E1-audit** reloads every CONVERGE run's persisted pipeline artefact and classifies each model recommendation by validator verdict and failing rule. **E2 (hallucination injection)** replaces the model with a scripted provider that returns an *oracle* plan derived from S6's real evidence plus exactly one hallucination from a twelve-class taxonomy (Table V); each case is executed end-to-end through both AI_ONLY and CONVERGE, isolating the validator's causal effect. **E3 (scalability)** generates datasets from 10^2 to 10^6 rows with a 10% mixed corruption rate and measures profiling, prompt size, validation, and execution separately.

VI. RESULTS

All 54 E1 runs completed successfully (success rate 1.00 for every cell); no provider timeouts or schema-parse failures occurred with constrained decoding. Table III and Fig. 2 summarise ground-truth outcomes.

A. RQ1: Does AI Improve Repair Quality?

Where the corruption requires a semantic decision and the model makes the right one, yes—decisively. On S5 (mixed date formats) the deterministic baseline, which by design refuses to choose a target format, leaves all three corrupted dates unrepaired (recall 0.00), whereas both AI arms choose `STANDARDIZE_DATE_FORMAT` with target `YYYY-MM-DD` in every repetition and restore all three cells exactly (recall 1.00, zero collateral damage).

TABLE III

E1 END-TO-END RESULTS (MEAN OVER 3 REPETITIONS; ± IS S.D.). *Recall*: CORRECTLY REPAIRED / INJECTED CORRUPTIONS (S2: FRACTION OF INJECTED DUPLICATES REMOVED). *C/I/U*: CORRECTLY REPAIRED, INCORRECTLY REPAIRED, UNREPAIRED CELLS. *Dmg*: ORIGINALLY CLEAN CELLS MODIFIED. *Issues*: COMPARATOR'S DETECTED ISSUE COUNT BEFORE→AFTER (AN OBSERVED SIGNAL, NOT CORRECTNESS). *Time*: STRATEGY WALL TIME IN SECONDS, INCLUDING THE LLM CALL.

Scen.	Arm	Recall	C/I/U	Dmg	FMR	Issues	Time
S1	SPARK_ONLY	1.00±0.00	3.0/0.0/0.0	0.0	0.000	1.0→0.0	14
	AI_ONLY	1.00±0.00	3.0/0.0/0.0	0.0	0.000	1.0→0.0	137
	CONVERGE	1.00±0.00	3.0/0.0/0.0	0.0	0.000	1.0→0.0	105
S2	SPARK_ONLY	1.00±0.00	–	0.0	0.000	2.0→0.0	12
	AI_ONLY	1.00±0.00	–	0.0	0.000	2.0→0.0	132
	CONVERGE	0.67±0.58	–	0.0	0.000	2.0→0.7	155
S3	SPARK_ONLY	0.00±0.00	0.0/0.0/3.0	0.0	0.000	2.0→2.0	12
	AI_ONLY	0.00±0.00	0.0/2.0/1.0	12.0	0.138	2.0→0.0	122
	CONVERGE	0.00±0.00	0.0/0.0/3.0	0.0	0.000	2.0→2.0	114
S4	SPARK_ONLY	0.00±0.00	0.0/0.0/2.0	0.0	0.000	1.0→1.0	10
	AI_ONLY	0.00±0.00	0.0/2.0/0.0	45.0	0.511	1.0→0.0	224
	CONVERGE	0.00±0.00	0.0/0.0/2.0	0.0	0.000	1.0→1.0	198
S5	SPARK_ONLY	0.00±0.00	0.0/0.0/3.0	0.0	0.000	1.0→1.0	13
	AI_ONLY	1.00±0.00	3.0/0.0/0.0	0.0	0.000	1.0→0.0	121
	CONVERGE	1.00±0.00	3.0/0.0/0.0	0.0	0.000	1.0→0.0	135
S6	SPARK_ONLY	0.23±0.00	3.0/0.0/8.0	0.0	0.000	5.0→4.0	14
	AI_ONLY	0.31±0.13	4.0/4.7/2.3	12.0	0.152	5.0→1.3	264
	CONVERGE	0.23±0.00	3.0/2.0/6.0	8.0	0.101	5.0→3.3	295

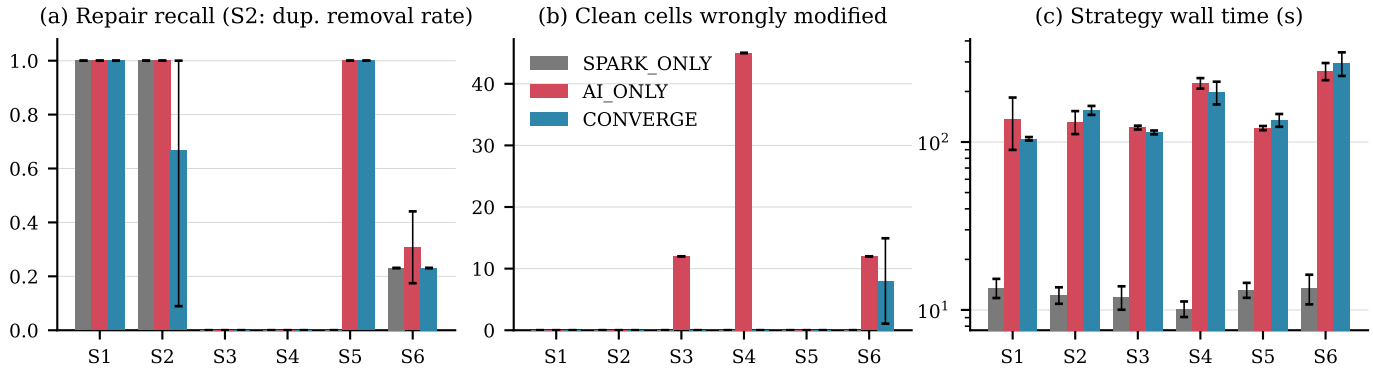


Fig. 2. E1 results by scenario and arm. (a) Ground-truth repair quality; (b) collateral damage to originally clean cells; (c) strategy wall time (log scale). AI is necessary for date and categorical semantics (S5), but unvalidated AI causes all of the large collateral damage (S3, S4).

Where the corruption is mechanically determined, AI adds nothing. On S1 all three arms reach recall 1.00; as discussed in §VII, this is partly an artefact of Spark's CSV writer, which trims whitespace on output, as the no-op control in E2 confirms. On S2 the baseline and AI_ONLY both remove both injected duplicates without losing an original row.

Where the corruption is information-theoretically unrecoverable, no arm succeeds. On S4 the true salaries cannot be derived from aggregates; AI_ONLY's mean imputation produces two *incorrect* repairs, while SPARK_ONLY and CONVERGE leave the cells missing.

Where the model's judgement is wrong, AI makes things worse. On S3 the model consistently proposed `NORMALIZE_CASE(lower)` on `department` rather than a canonicalising `STANDARDIZE_CATEGORY`. Lowercasing does unify the variant groups, but toward a value (`engineering`) that disagrees with the twelve correct cells. On S6 the model's plan varied between repetitions (consistency 0.67) and achieved mean recall 0.31 ± 0.13 under AI_ONLY, versus 0.23 for the baseline.

B. RQ2: Does Validation Reduce Hallucination-Driven Errors?

Collateral damage (Fig. 2b) answers RQ2 most directly. Unvalidated AI modified **12** clean cells on S3, **45** on S4 (FMR 0.51: the model invented whitespace and casing problems on `name`, `department` and `city` for which the profiler had produced no evidence, and lowercased all three columns), and **12** on S6. Under CONVERGE the damage was **0** on S4, where all three fabricated recommendations were rejected for lack of evidence (`evidence_exists`) or an ill-typed parameter, and **0** on S3. Summed over the six scenarios, mean collateral damage per run fell from 69 to 8 cells (−88%),

while every correct repair the model produced on S1 and S5 was preserved. The audit below shows, however, that the S3 outcome must be attributed with care: two of the three S3 rejections stem from a validator defect, and with that defect corrected the S3 recommendation would have been *approved* and would have caused the same 12-cell damage. Under the corrected validator the reduction is therefore $69 \rightarrow 20$ cells (-71%)—still substantial, and entirely due to evidence checks that worked as designed (S4).

The comparator's observed signals (Table III, *Issues*) are instructive: by that measure AI_ONLY looks *best* on S3 and S4 (all detected issues resolved, $2 \rightarrow 0$ and $1 \rightarrow 0$), precisely where it inflicted the most damage. Lowercasing every value removes the variant groups the profiler detects, and imputation removes the null count. This vindicates the decision to never merge observed issue counts with ground-truth correctness (ADR-009): *an AI cleaning system evaluated only by downstream data-quality checks can appear to succeed while corrupting the data.*

C. Recommendation-Level Audit

Across the 18 CONVERGE runs the model produced 35 recommendations (Table IV). The validator approved 15 (13 SAFE, 2 REVIEW) and rejected 20: 6 for absent evidence (fabricated issues), 8 for contradicted claims, and 6 for an ill-typed parameter (`fill_value: null` on a mean imputation). Manual inspection of every verdict against ground truth reveals three distinct failure modes of the *validator*, each of which we consider a contribution of the audit:

1) **Evidence-ambiguity false rejections (3 of 20).** When the profiler emits several evidence items with the same issue type and column (two variant groups for `department`; two duplicate groups on S2), the claim check compares the model's claimed values against only the *first* matching item. A correct claim about the second group (e.g., duplicate pair R011/R016, which the ground-truth manifest confirms) is then reported as a contradiction. An offline re-validation with a corrected "any-matching-item" rule flips exactly these three verdicts. The consequences are mixed: on S2 the false rejection cost one repetition both duplicate removals, whereas on S3 it *accidentally* blocked a harmful lowercasing—a reminder that a validator's apparent wins must be audited as carefully as its losses.
2) **Over-strict optional parameters (6 of 20).** Every `FILL_MISSING` recommendation passed `fill_value: null` for an optional parameter and was rejected on type; this is conservative but harmless here, since any imputation is an incorrect repair under exact ground truth.
3) **Evidence-type conflation (2 approvals).** On S6, `NORMALIZE_CASE(lower)` on `name` was *approved* because the catalog lets `whitespace_or_casing` evidence justify a case change; the profiler had flagged only whitespace. All 12 clean names were lowercased. This is the source of CONVERGE's residual damage on S6 and shows that *the validator is only as strong as the evidence requirements declared in its catalog*.

TABLE IV
E1 AUDIT OF ALL 35 LLM RECOMMENDATIONS UNDER CONVERGE.

Validator outcome	n	Ground-truth assessment
Approved, SAFE tier	13	11 beneficial, 2 harmful (conflation)
Approved, REVIEW tier	2	beneficial (S2 dedup)
Rejected: no evidence	6	all fabricated issues (S4)
Rejected: contradicted claim	8	5 genuinely false, 3 ambiguity
Rejected: parameter type	6	`fill_value: null`

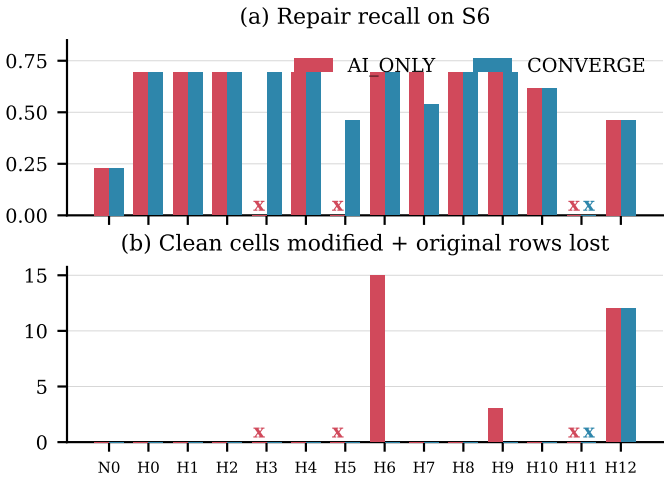


Fig. 3. E2 per-case outcomes on S6. An "x" marks an aborted run (no output).

Conversely, the validator also rejected a recommendation that would have been beneficial: on S6 repetition 0 the model proposed the correct date standardisation but claimed `{YYYY-MM-DD: 14}` for a column whose real histogram contained two formats; the contradicted claim cost three correct repairs. We regard this as correct behaviour—a recommendation whose stated evidential basis is false should not execute unattended—but it quantifies the conservativeness cost of evidence validation.

D. RQ3: Causal Effect of Validation per Hallucination Class (E2)

E2 removes model stochasticity: a scripted provider returns an oracle plan derived from S6's real evidence (`TRIM_WHITESPACE`, two `STANDARDIZE_CATEGORY` mappings, `STANDARDIZE_DATE_FORMAT`, `FILL_MISSING(median)`) and one injected hallucination. Hallucinations either *append* an extra recommendation (+) or *substitute* for their correct counterpart ($\leftrightarrow$), as a real model would. Table V and Fig. 3 report the outcomes.

The thirteen cases fall into five groups.

(i) Caught by structure alone (H1, H2). Invented and forbidden operation names are dropped by AI_ONLY's structural filter as well; both arms yield the oracle result. A closed operation catalog is, by itself, an effective first line of defence—an argument for recommendation over code generation.

TABLE V

E2 HALLUCINATION INJECTION ON S6 (13 CORRUPTIONS; DETERMINISTIC, SO NO VARIANCE). *Mode*: + APPENDED TO THE ORACLE PLAN, ↔ SUBSTITUTED FOR ITS CORRECT COUNTERPART. *Verdict*: THE VALIDATOR'S DECISION ON THE INJECTED RECOMMENDATION AND THE FIRST FAILING RULE. *Dmg*: ORIGINALLY CLEAN CELLS MODIFIED PLUS ORIGINAL ROWS LOST. "RUN ABORTED": EXECUTOR FAIL-FAST STOP, NO OUTPUT WRITTEN.

				AI_ONLY		CONVERGE	
ID	**Hallucination class**	**Mode**	**Verdict**	**Recall**	**Dmg**	**Recall**	**Dmg**
N0	No-op control		–	0.23	0	0.23	0
H0	Oracle control		–	0.69	0	0.69	0
H1	Invented operation	+	rej. (unknown op)	0.69	0	0.69	0
H2	Forbidden operation	+	rej. (forbidden)	0.69	0	0.69	0
H3	Phantom column	+	rej. (no such column)	*run aborted*		0.69	0
H4	Type-incompatible target	+	rej. (dtype)	0.69	0	0.69	0
H5	Out-of-domain parameter	↔	rej. (parameters)	*run aborted*		0.46	0
H6	Smuggled parameter	+	rej. (parameters)	0.69	15	0.69	0
H7	Fabricated value in mapping	↔	rej. (claim contradicted)	0.69	0	0.54	0
H8	Contradicted numeric claim	↔	rej. (claim contradicted)	0.69	0	0.69	0
H9	Unsupported destructive op	+	rej. (no evidence)	0.69	3	0.69	0
H10	Semantically wrong, evidenced	↔	**approved**	0.62	0	0.62	0
H11	Uncoercible constant	↔	**review**	*run aborted*		*run aborted*	
H12	Evidence-type conflation	+	**approved**	0.46	12	0.46	12

(ii) Availability failures prevented (H3, H5). A phantom column or an out-of-domain parameter is caught by the executor's defence-in-depth checks under AI_ONLY, but fail-fast semantics then *abort the entire run*: one hallucinated recommendation discards every correct repair. CONVERGE rejects the bad recommendation up front and executes the rest (H3: recall 0.69; H5: 0.46, since the substituted date operation is lost).

(iii) Silent corruption prevented (H6, H9). These are the most dangerous cases, because AI_ONLY *succeeds*. H6 smuggles an extra `sql` key into an otherwise valid `NORMALIZE_CASE` on `city`; the executor ignores unknown keys, upper-cases the column, and damages 15 clean cells. H9 proposes `REMOVE_DUPLICATES` on criteria {`department`, `city`}, which no evidence supports; AI_ONLY silently deletes three *original* employee records. CONVERGE rejects both (unexpected parameter; no exact-criteria duplicate evidence) with zero damage.

(iv) Conservativeness costs (H7, H8, H4). H7 adds one fabricated key (`Engg.`) to an otherwise correct mapping. The fabricated key is harmless at execution time, so AI_ONLY keeps all repairs (recall 0.69), whereas CONVERGE rejects the whole recommendation and loses two correct category repairs (0.54). This is the only case in which unvalidated AI outperforms validated AI. H8 (a contradicted null count) is rejected, leaving two salaries missing instead of incorrectly imputed: equal recall, but 2 fewer incorrect repairs. H4 (trimming an integer column) is rejected on dtype but would have been a no-op.

(v) Beyond the reach of validation (H10, H11, H12). H10 maps the *real* variant `mARKETING` to `Sales`. Every checkable predicate holds, the validator approves it, and both arms produce the same incorrect repair. H12 reproduces the evidence-type conflation observed in E1: whitespace evidence licenses a lowercase of `name`, which is approved and damages 12 clean cells in both arms. H11 proposes a constant `"unknown"` for an integer column; the parameter is a well-typed string, so the validator admits it at the REVIEW tier, and the executor's coercion check then aborts the run in *both* arms. H11 identifies a concrete gap—type-coercibility of constant fill values is checkable from the profile and should be validated—whereas H10 is the irreducible class discussed in §VII.

Summary. Of the eleven checkable classes (H1–H9, H11, H12), the validator rejected nine. It turned two run aborts into successful partial cleaning and prevented 15 corrupted cells and 3 lost records. It cost two correct repairs in one case. The no-op control N0 (recall 0.23) confirms that three whitespace repairs are performed by Spark's CSV writer irrespective of the plan.

E. RQ4: Latency, Reliability, and Scalability

Latency. In E1, SPARK_ONLY ran in 10–14 s per strategy invocation, while the AI arms took 105–295 s (Table III). The difference is almost entirely model inference: the AI-analysis stage of CONVERGE averaged 161 ± 68 s per run (range 93–324 s) on the 4-core CPU, growing with the number of evidence items and hence with the length of the generated plan (S1, one item: 95 s; S6, five items: 278 s). Validation, including persistence of the audit record, took 61–130 ms per run, and the validator's checks alone take well under a millisecond (below). There is no systematic latency difference between AI_ONLY and CONVERGE; observed differences are within the run-to-run variation of CPU inference. The validation boundary is therefore effectively free relative to the model call.

Reliability. All 54 runs succeeded; schema-constrained decoding eliminated the parse failures and timeouts seen in the uncontrolled pilot. Recommendation consistency (mean pairwise Jaccard similarity of (o, C_r) sets across repetitions) was 1.00 on S1–S5 for both AI arms and 0.67 on S6, where the model varied its plan (e.g., lowercasing `name` in some repetitions and `department` in another) even at temperature 0. Temperature-0 decoding on quantised CPU inference is

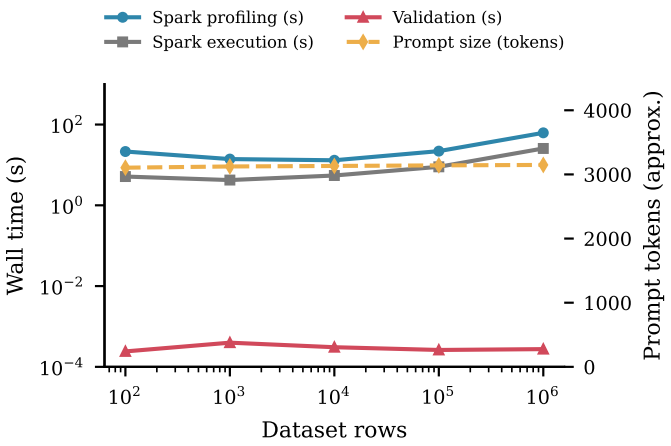


Fig. 4. E3: Spark profiling and execution scale with data volume, whereas the AI-facing prompt and validation cost are independent of it (log–log; prompt size on the right axis).

TABLE VI
E3 CONTROL-PLANE VS. DATA-PLANE COST BY DATASET SIZE.

Rows	Profile (s)	Execute (s)	Prompt (chars)	Evid.	Validate (ms)
10^2	21.5	5.2	12,423	6	0.24
10^3	14.0	4.2	12,498	6	0.40
10^4	13.0	5.5	12,530	6	0.31
10^5	22.0	9.0	12,564	6	0.26
10^6	62.3	25.6	12,593	6	0.27

therefore not a reproducibility guarantee for multi-issue inputs, which strengthens the case for a deterministic gate.

Scalability. Fig. 4 and Table VI report E3. Across four orders of magnitude ($10^2 \rightarrow 10^6$ rows, 6.4 KB→66 MB), Spark profiling grew from 14 s to 62 s and plan execution from 4 s to 26 s (the 10^2-row profiling time includes JVM warm-up). Over the same range the AI-facing prompt grew by only 1.4% (12,423→12,593 characters, ≈3.1k tokens; the growth is the extra digits in larger counts), the evidence bundle stayed at six items, and validating the six-recommendation plan took 0.24–0.40 ms regardless of size. This confirms the structural claim of §IV: in CONVERGE the expensive, probabilistic component is invoked on $O(m + |E|)$ input, once per dataset, while the row-proportional work stays in the deterministic engine that was designed for it. A single model call of fixed cost can thus govern a cleaning job of arbitrary size, which is what makes LLM assistance economically compatible with big data workloads.

VII. DISCUSSION

A. What the Validation Boundary Buys

The results support a nuanced reading of RQ2. The validator does not make the model *smarter*: on the scenarios where the model's output was already correct, AI_ONLY and CONVERGE produce identical datasets. What the validator buys is a *tighter worst case*. In E2, nine of eleven checkable hallucination classes were converted from potential data-integrity incidents (collateral modification, lost records, or an aborted run) into auditable rejections with a named rule, while the remaining correct recommendations still executed; in E1, the model's fabricated issues on S4 were rejected before they could corrupt 45 cells. This is precisely the property that big data operators require before delegating any decision to a component they cannot formally verify: for every hallucination class the validator can check, the blast radius of an error shrinks from *doing the wrong thing at scale* to *not doing* something—and the classes it cannot yet check are named and measured rather than unknown.

The audit trail matters as much as the verdict. Because every rejection records the rule that fired and the profiler fact it contradicted, operators can distinguish "the model proposed something unsupported" from "the model misread a statistic" from "the model attempted a forbidden action"—three failure modes that warrant different responses (prompt revision, model replacement, and security review, respectively).

B. What Validation Cannot Buy

Two classes of error lie provably outside the reach of any evidence-based validator.

Semantically wrong but evidentially consistent recommendations (H10) satisfy every checkable predicate of §III: the operation is legal, the column exists, the mapping key really occurs in the data. Only a judgement about the *meaning* of "Marketing" versus "Sales" separates it from a correct repair. Such errors can be reduced but not eliminated by stronger structural rules—e.g., requiring every mapping to stay within one variant group (a normalisation-equivalence constraint), which would catch H10 specifically—or by consensus across models; ultimately they motivate the human approval boundary. The S3 behaviour in E1 is a natural instance: lowercasing *does* unify the evidenced variants, and is rejected only by accident.

Two further gaps are *not* fundamental and point to concrete fixes. Evidence-type conflation (H12, and the S6 approvals in E1) arises because the catalog lets one evidence type (`whitespace_or_casing`) license two semantically different operations; splitting the issue type, or requiring that a casing operation be justified by a variant group, closes it. The coercibility gap (H11) is closed by checking constant fill values against the column's semantic type. Likewise, the evidence-ambiguity defect found in the audit is fixed by accepting a claim if it matches *any* candidate evidence item. All three are small, deterministic changes—the architecture makes such failures *diagnosable and fixable*, which is itself a property unvalidated pipelines lack.

Information-theoretically unrecoverable repairs are the second class. A missing salary cannot be restored to its true value from aggregate statistics; any imputation (mean, median, mode) is, by construction, an *incorrect repair* under exact-match ground truth. S4's results should be read in this light: the correct behaviour for a cleaning system is not to maximise recall there, but to make the imputation explicit, review-

able, and reversible—which CONVERGE does by capping `FILL_MISSING` at the REVIEW tier.

C. The Conservativeness Cost

Validation is not free. A single fabricated key in an otherwise-correct category mapping (H7) causes the *entire* recommendation to be rejected, forfeiting its correct repairs. We consider this the right default for an unattended pipeline—partial trust in a recommendation whose evidence claims are known to be partly false is unjustified—but it suggests a refinement: *repairable rejections*, in which the validator returns a minimally edited recommendation (here, the mapping with the fabricated key removed) for explicit approval. Similarly, short-circuiting on the first failing rule produces the most actionable explanation but hides additional violations; a "report all" mode would aid prompt engineering.

D. Small Local Models as Recommenders

The use of a 3B-parameter model quantised to 4 bits on a laptop CPU is deliberate. It reflects the deployment constraint that motivates CONVERGE in the first place—regulated data that may not leave the cluster—and it is the regime in which structured-output failures are most frequent. The architecture's value is largest exactly here: a weak model behind a strong validator is safe to deploy, whereas a strong model without one is merely *usually* safe. Schema-constrained decoding was essential: in the earlier pilot without it, the same model returned `parameters` as a list and `evidence_refs` as an object, failing strict parsing.

E. Sink-Side Normalisation: A Methodological Lesson

E2's no-op control revealed that Spark's CSV writer, with its default options, removes leading and trailing whitespace from every string field on output. Whitespace corruptions are therefore "repaired" by the storage layer regardless of the cleaning decision, and S1 cannot discriminate between arms. This is not an artefact of CONVERGE but of the execution substrate, and it generalises: when evaluating AI-assisted cleaning, the evaluator must include a no-op baseline to separate the effect of the decision layer from implicit normalisation in readers and writers. We report S1 for completeness and exclude it from comparative claims.

VIII. Threats to Validity

Internal validity. All arms share every component except the decision layer, and the evaluator is strategy-agnostic by construction (a test asserts it contains no strategy-specific branching). The approval policy is automated; a human approver could reject correct recommendations or accept wrong ones. Temperature-0 decoding reduces but does not remove stochasticity under quantised CPU inference.

Construct validity. Exact-match ground truth penalises semantically acceptable alternatives (e.g., a different but reasonable imputation). Before/after issue counts are reported separately precisely because they measure *absence of detected issues*, not correctness. Repair precision is not observable and is not reported.

External validity. The reference table has 15 rows and 7 columns and covers five corruption types; real datasets exhibit richer errors (functional-dependency violations, cross-column inconsistencies, typos). We evaluated one local model; larger or remote models (e.g., Gemini, supported by the implementation) may differ in recall and failure profile. Spark ran in `local[*]` mode; the control-plane scalability result (E3) is structural, but absolute Spark timings on a cluster will differ.

Conclusion validity. With three repetitions per cell we report means and standard deviations but do not claim statistical significance; the E2 study is deterministic and its conclusions do not depend on sampling.

IX. Future Directions

Our results suggest that the most productive path for integrating generative AI with traditional big data pipelines is not to make models execute, but to make their proposals *verifiable*. We highlight seven directions.

1) **Richer evidence languages.** Extending the profiler to emit functional-dependency, cross-column, and distributional-drift evidence (in the spirit of Deequ [21] and CFDs [16]) would let the catalog grow while keeping every operation checkable.
2) **Semantic validators.** Normalisation-equivalence constraints, reference-vocabulary lookups, and multi-model agreement can narrow the H10 gap without surrendering determinism.
3) **Repairable rejections and dialogue.** Feeding validator explanations back to the model, Reflexion-style [26], turns rejections into a bounded self-correction loop whose output is *re-validated*, preserving complete mediation.
4) **Placement beyond cleaning.** The same recommend–validate–execute pattern applies before ingestion (schema inference), during processing (partitioning and join-order hints validated against cost models), and after processing (natural-language insights validated against computed aggregates).
5) **Cluster-scale evaluation.** Because the prompt is $O(m + |E|)$, a single model call can govern a petabyte-scale Spark job; the open question is how to sample and bound evidence for very wide or high-cardinality schemas.
6) **Cost and carbon accounting.** Exposing token usage through the provider interface would allow cost-aware placement between local SLMs and remote LLMs, guided by scaling behaviour [34].
7) **Adversarial robustness.** Evidence contains raw cell values; a malicious cell could attempt indirect prompt injection [29]. Because the validator treats all output as untrusted, the architecture already bounds the damage, but systematic red-teaming of the evidence channel is needed.

X. Conclusion

We presented CONVERGE, a validation-guided architecture for integrating language models into Spark data-cleaning pipelines, in which the model recommends, an independent

deterministic validator decides against Spark-computed evidence, and Spark executes an immutable, fail-fast plan. Under controlled, ground-truth conditions with a local 3B-parameter model, AI-assisted cleaning repaired a corruption class that a principled deterministic baseline must leave untouched (mixed date formats, recall $0 \rightarrow 1$), but unvalidated AI also fabricated issues and destroyed up to 45 clean cells in a single run—damage that standard before/after quality checks scored as a success. The validation boundary cut collateral damage by 71–88%, rejected nine of the eleven checkable hallucination classes we injected—turning silent corruption, deleted records and aborted runs into auditable rejections—and did so at a sub-millisecond control-plane cost that does not grow with data volume. Validation is not a substitute for semantic correctness: semantically wrong but evidentially consistent recommendations, catalog-level evidence conflation, and unrecoverable imputations remain, and they define where human review and future semantic validators must focus. The broader lesson for the future of big data systems is architectural: generative models are most valuable to traditional pipelines when they are confined to proposing, and every proposal is checked by the deterministic machinery those pipelines already trust.

Artifact Availability

The CONVERGE implementation, controlled-corruption generator, ground-truth manifests, experiment harness, raw per-run results, and the scripts that produced every figure and table in this paper are contained in the project repository (`src/converge`, `experiments/`, `paper/scripts/`). All 510 automated tests pass.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[3] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] E. Rahm and H. H. Do, "Data cleaning: Problems and current approaches," *IEEE Data Engineering Bulletin*, vol. 23, no. 4, pp. 3–13, 2000.

[5] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2016, pp. 2201–2206.

[6] I. F. Ilyas and X. Chu, *Data Cleaning*. ACM Books, 2019.

[7] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[8] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Large language models as data preprocessors," *arXiv preprint arXiv:2308.16361*, 2023.

[9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[10] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, 2025.

[11] N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-SQL capabilities of large language models," *arXiv preprint arXiv:2204.00498*, 2022.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[13] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[14] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[15] J. M. Hellerstein, "Quantitative data cleaning for large databases," *United Nations Economic Commission for Europe (UNECE)*, 2008.

[16] W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, "Conditional functional dependencies for capturing data inconsistencies," *ACM Transactions on Database Systems*, vol. 33, no. 2, pp. 6:1–6:48, 2008.

[17] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1190–1201, 2017.

[18] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 948–959, 2016.

[19] M. Mahdavi, Z. Abedjan, R. C. Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang, "Raha: A configuration-free error detection system," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019, pp. 865–882.

[20] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 993–1004, 2016.

[21] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[22] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. Machine Learning and Systems (MLSys)*, 2019.

[23] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[24] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, "Wrangler: Interactive visual specification of data transformation scripts," in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2011, pp. 3363–3372.

[25] J. Li *et al.*, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[26] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[27] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[28] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[29] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[30] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[31] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[32] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[33] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[34] J. Kaplan *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.